



TOPORAY

Radiación simple

DOCUMENTACIÓN DEL SISTEMA

MANUAL DEL PROGRAMADOR

Arquitectura, cálculo, persistencia, mantenimiento y despliegue

Campo	Información
Versión del documento	1.4
Versión del software	TOPORAY 1.0
Fecha de actualización	15 de septiembre de 2026
Producto	TOPORAY · Radiación simple
Estado	Documento técnico vigente

Documento preparado para consulta académica, operación controlada y mantenimiento del sistema.

Control documental

Este manual documenta la implementación de TOPORAY versión 1.0. Está dirigido a desarrolladores responsables de revisar, mantener, probar o desplegar la aplicación.

Versión	Fecha	Descripción	Estado
1.0	28/07/2026	Emisión inicial del manual	Histórico
1.1	19/08/2026	Actualización funcional y guía práctica	Histórico
1.2	02/09/2026	Orden por barra lateral y numeración de tablas y figuras	Histórico
1.3	15/09/2026	Exportación TXT PENZD, compatibilidad con Civil 3D y aviso inicial	Histórico
1.4	15/09/2026	Aviso obligatorio antes de cada descarga de coordenadas TXT	Vigente

Contenido

1. Descripción técnica	3
2. Estructura del proyecto	4
3. Estructura HTML y navegación	5
4. Sistema visual y adaptación responsive	6
5. Modelo de estado	7
6. Proyectos: persistencia e historial	8
7. Zonas: geometría y referencias	9
8. Puntos: validación y ciclo de vida	11
9. Plano: renderizado cartesiano	12
10. Cálculos: conversión angular, rumbo y proyecciones	14
11. Importar y exportar: formatos y salidas	16
12. Configuración: informe técnico y escala de dibujo	17
13. Créditos: estructura institucional	18
14. Eventos, accesibilidad y comportamiento móvil	19
15. Pruebas y control de calidad	20
16. Mantenimiento, seguridad y limitaciones	21

1. Descripción técnica

TOPORAY es una aplicación web estática de una sola página. No requiere compilador, gestor de paquetes, servidor de aplicaciones ni base de datos. La lógica se ejecuta íntegramente en el navegador.

Tabla 1. Capas tecnológicas de TOPORAY.

Capa	Tecnología	Responsabilidad
Estructura	HTML5	Navegación, vistas, formularios, tablas, diálogos y canvas.
Presentación	CSS3	Temas, distribución, tablas fijas y reglas responsive.
Lógica	JavaScript ES6+	Estado, validación, cálculo, dibujo, persistencia y exportación.
Gráfica	Canvas 2D	Plano cartesiano interactivo y plano técnico exportable.
Persistencia	localStorage	Proyecto activo, proyectos nombrados y selección actual.
Intercambio	CSV/TXT/PNG/HTML	Importación, exportación e informe imprimible.

Principio de operación

El objeto global state es la fuente de verdad. `update()` calcula el levantamiento, sincroniza salidas, redibuja el canvas y conserva el estado local.

Compatibilidad objetivo

- Navegadores modernos con Canvas 2D, dialog, structuredClone, Blob y localStorage.
- Diseño adaptable desde 320 px de ancho.
- Ejecución directa desde un servidor HTTP estático o GitHub Pages.
- No existe sincronización multiusuario ni autenticación.

2. Estructura del proyecto

Tabla 2. Estructura de archivos del proyecto.

Ruta	Contenido
index.html	Punto de entrada y estructura completa de la interfaz.
styles.css	Variables, componentes, layout, temas, responsive e impresión.
app.js	Estado, motor topográfico, canvas, proyectos y productos de salida.
assets/toporay-*.svg	Identidad visual y favicon.
assets/report-template.png	Plantilla institucional utilizada por el informe.
README.md	Instrucciones básicas de ejecución y resumen funcional.
output/pdf	Manuales finales generados.
tmp/pdfs	Activos y scripts temporales de documentación y revisión.

Ejecución local

Comando

```
python -m http.server 5175  
Después, abrir http://127.0.0.1:5175/.
```

Despliegue estático

- Publique index.html, styles.css, app.js y assets sin cambiar sus rutas relativas.
- Configure index.html como documento de entrada.
- Use HTTPS en producción para un origen estable de localStorage.
- Evite cambiar el origen o la ruta publicada sin planificar la migración de proyectos del navegador.

3. Estructura HTML y navegación

Las vistas se mantienen en el mismo documento y se identifican mediante data-view y data-view-panel. switchView() activa el panel solicitado y actualiza el estado visual del menú.

Código 1. Navegación principal - index.html

```
21 <nav class="sidebar-nav">
22   <button class="nav-button" type="button" data-view="projects" title="Proyectos">
23     <span class="nav-icon" aria-hidden="true"></span><span class="sidebar-label">Proyectos</span>
24   </button>
25   <button class="nav-button" type="button" data-view="zones" title="Zonas">
26     <span class="nav-icon" aria-hidden="true"></span><span class="sidebar-label">Zonas</span>
27   </button>
28   <button class="nav-button" type="button" data-view="points" title="Puntos">
29     <span class="nav-icon" aria-hidden="true"></span><span class="sidebar-label">Puntos</span>
30   </button>
31   <button class="nav-button is-active" type="button" data-view="plan" title="Plano">
32     <span class="nav-icon" aria-hidden="true"></span><span class="sidebar-label">Plano</span>
33   </button>
34   <button class="nav-button" type="button" data-view="calculations" title="Cálculos">
35     <span class="nav-icon" aria-hidden="true"></span><span class="sidebar-label">Cálculos</span>
36   </button>
37   <button class="nav-button" type="button" data-view="files" title="Importar y exportar">
38     <span class="nav-icon" aria-hidden="true"></span><span class="sidebar-label">Importar y exportar</span>
39   </button>
40   <button class="nav-button" type="button" data-view="settings" title="Configuración">
41     <span class="nav-icon" aria-hidden="true"></span><span class="sidebar-label">Configuración</span>
42   </button>
43   <button class="nav-button" type="button" data-view="credits" title="Créditos">
44     <span class="nav-icon" aria-hidden="true"></span><span class="sidebar-label">Créditos</span>
45   </button>
46 </nav>
```

Figura 1. Fragmento de código: Navegación principal declarada en index.html, líneas 21–43.

Tabla 3. Correspondencia entre la barra lateral y los módulos del código.

Orden	Vista	Componentes principales
1	Proyectos	renderProjectList(), saveNamedProject(), loadProjectByName().
2	Zonas	renderZones(), createZone() y referencias de cálculo.
3	Puntos	renderPoints(), pointForm y validación de observaciones.
4	Plano	drawPlot(), setQuickMode() y eventos del canvas.
5	Cálculos	computeSurvey() y renderCalculations().
6	Importar y exportar	importCsv(), exportCsv(), exportCoordinatesTxt() y printReport().
7	Configuración	syncControls(), drawingScaleDetails() y preferencias visuales.
8	Créditos	Contenido institucional estático en #view-credits.

Convención de navegación

Los atributos data-view y data-view-panel conservan el mismo nombre lógico en escritorio y móvil. app.js se carga al final de body para que todos los elementos estén disponibles antes de enlazar eventos.

Aviso de consulta del manual

#manualNoticeDialog se abre mediante openManualNotice(). La aceptación inicial se conserva en sessionStorage con la clave toporay-manual-notice-v1. requestCoordinatesTxtExport() fuerza nuevamente el diálogo en cada intento de exportación y confirmManualNotice() ejecuta exportCoordinatesTxt() solo después de la confirmación Descargar TXT.

4. Sistema visual y adaptación responsive

styles.css define los temas mediante variables y conserva una interfaz operacional de alta densidad. El tema claro reemplaza los mismos tokens sin duplicar los componentes.

Código 3. Adaptación móvil - styles.css

```
1381 border: 1px solid var(--line-strong);
1382 border-left: 4px solid var(--accent);
1383 border-radius: 4px;
1384 background: var(--panel-raised);
1385 box-shadow: var(--shadow);
1386 opacity: 0;
1387 pointer-events: none;
1388 transform: translateY(8px);
1389 transition: opacity 150ms ease, transform 150ms ease;
1390 }
1391
1392 .toast.is-visible {
1393   opacity: 1;
1394   transform: translateY(0);
1395 }
1396
1397 .toast.is-error {
1398   border-left-color: var(--danger);
1399 }
1400
1401 .is-hidden {
1402   display: none !important;
1403 }
1404
1405 @media (max-width: 980px) {
1406   .app-sidebar {
1407     width: min(280px, 82vw);
1408     transform: translateX(-102%);
1409   }
1410
1411   .app-sidebar.is-mobile-open {
1412     transform: translateX(0);
1413   }
1414 }
```

Figura 2. Fragmento de código: Reglas de adaptación para pantallas pequeñas.

Tabla 4. Criterios de presentación y adaptación responsive.

Criterio	Implementación
Layout	Barra lateral fija, cabecera, panel principal y navegación móvil.
Plano	Canvas cuadrado con dimensiones estables y relación de aspecto controlada.
Tablas	Desplazamiento horizontal; zona, punto y acciones permanecen visibles.
Puntos de quiebre	980 px para menú móvil y 650 px para reorganización compacta.
Impresión	Fondo blanco, texto oscuro y ocultamiento de comandos de interacción.

5. Modelo de estado

Código 4. Estado inicial - app.js

```
205     description: "",
206   };
207 }
208
209 function createBlankState(projectName = "Levantamiento sin nombre") {
210   const mainZone = createZone({ id: makeId("zone"), name: "Zona principal", color: ZONE_COLORS[0] });
211   return {
212     version: 2,
213     projectName,
214     modifiedAt: new Date().toISOString(),
215     projectMeta: { responsable: "", surveyDate: "", reviewer: "", printDate: todayIsoDate() },
216     station: { east: 1000, north: 1000 },
217     mode: "radiacion",
218     showPoints: true,
219     showLines: true,
220     showGrid: true,
221     showZoneNames: true,
222     showPointNumbers: true,
223     units: "m",
224     gridSize: 0,
225     appearance: "dark",
226     axisDecimals: 0,
227     view: { zoom: 1, panEast: 0, panNorth: 0 },
228     drawingScale: { paper: "A3", orientation: "horizontal", mode: "auto", denominator: 500, customWidth: 420, customHeight: 297 },
```

Figura 3. Fragmento de código: Estado inicial normalizado de un levantamiento.

Tabla 5. Propiedades principales del estado de la aplicación.

Propiedad	Tipo	Responsabilidad
projectName	string	Nombre visible y clave propuesta al guardar.
station	{east,north}	Coordenadas del BM fijo.
mode	string	Base de cálculo: radiación o poligonal.
zones	array	Objetos geométricos, estilo, referencia y visibilidad.
observations	array	GMS, distancia, descripción, UID y zona de cada punto.
view	object	Zoom y desplazamiento del canvas.
drawingScale	object	Papel, orientación, modo y denominador.
show*	boolean	Preferencias de puntos, líneas, cuadrícula y rótulos.

migrateState() completa propiedades faltantes y normaliza proyectos guardados por versiones anteriores. Cada zona y punto usa un UID para conservar identidad durante reordenamientos.

6. Proyectos: persistencia e historial

Código 9. Persistencia local - app.js

```
2075 function writeProjects(projects) {
2076   localStorage.setItem(PROJECTS_KEY, JSON.stringify(projects));
2077 }
2078
2079 function saveLocalState() {
2080   localStorage.setItem(STORAGE_KEY, JSON.stringify(state));
2081   const normalizedName = normalizeProjectName(state.projectName);
2082   if (loadedProjectName && normalizedName === loadedProjectName) {
2083     const projects = loadProjects();
2084     state.modifiedAt = new Date().toISOString();
2085     projects[loadedProjectName] = clone({ ...state, projectName: loadedProjectName, modifiedAt: state.modifiedAt });
2086     writeProjects(projects);
2087     localStorage.setItem(CURRENT_PROJECT_KEY, loadedProjectName);
2088     els.saveState.textContent = "Guardado automáticamente";
2089   } else if (loadedProjectName) {
2090     els.saveState.textContent = "Nombre nuevo: pulse Guardar";
2091   } else {
2092     els.saveState.textContent = "Cambios locales";
2093   }
2094 }
2095
2096 function saveNamedProject() {
2097   const name = normalizeProjectName(els.projectName.value);
2098   const projects = loadProjects();
2099   const isDifferentProject = loadedProjectName !== name;
2100   if (isDifferentProject && projects[name] && !window.confirm(`Ya existe un levantamiento llamado "${name}". ¿Desea reemplazarlo?`)) return;
2101   state.projectName = name;
2102   state.modifiedAt = new Date().toISOString();
2103   projects[name] = clone(state);
2104   writeProjects(projects);
2105   loadedProjectName = name;
2106   localStorage.setItem(CURRENT_PROJECT_KEY, name);
2107   localStorage.setItem(STORAGE_KEY, JSON.stringify(state));
}
```

Figura 4. Fragmento de código: Persistencia del estado y proyectos.

Tabla 6. Claves utilizadas en el almacenamiento local.

Clave	Contenido
levantamientos-topograficos-v1	Estado activo serializado.
levantamientos-topograficos-projects-v1	Mapa de proyectos guardados por nombre.
levantamientos-topograficos-current-project-v1	Nombre del proyecto actualmente abierto.

saveLocalState() se ejecuta durante update(). Si el nombre coincide con el proyecto cargado, también actualiza su copia nombrada. pushHistory(), undo() y redo() conservan instantáneas JSON; HISTORY_LIMIT limita la pila a 60 estados.

Riesgo operativo
localStorage depende del origen, perfil y dispositivo. No existe respaldo de servidor. La exportación CSV es el mecanismo de respaldo portátil.

7. Zonas: geometría y referencias

Código 7. Área y perímetro - app.js

```
656     if (current.length) segments.push(current);
657     return segments;
658 }
659
660 function polygonArea(points) {
661     if (points.length < 3) return 0;
662     let sum = 0;
663     for (let index = 0; index < points.length; index += 1) {
664         const a = points[index];
665         const b = points[(index + 1) % points.length];
666         sum += a.east * b.north - b.east * a.north;
667     }
668     return Math.abs(sum) / 2;
669 }
670
671 function perimeter(points, close) {
672     if (points.length < 2) return 0;
673     let total = 0;
674     for (let index = 1; index < points.length; index += 1) {
675         total += Math.hypot(points[index].east - points[index - 1].east, points[index].north - points[index - 1].north);
676     }
677 }
```

Figura 5. Fragmento de código: Cálculo de área por determinantes y longitud por segmentos.

Tabla 7. Condiciones geométricas por tipo de zona.

Tipo de zona	Condición de completitud	Métrica
Polígono	Tres o más puntos válidos, cerrado, sin cruces, duplicados ni vacíos internos.	Área y perímetro.
Línea	Dos o más puntos válidos y continuidad de segmentos.	Longitud.
Puntos	Al menos un punto válido.	Conteo y coordenadas.

- polygonArea() aplica la fórmula del cordón o shoelace y devuelve el valor absoluto entre 2.
- perimeter() suma distancias euclidianas entre pares consecutivos y agrega el cierre cuando corresponde.
- polygonHasCrossings() compara segmentos no adyacentes mediante orientación.
- buildSegments() separa tramos cuando existe una fila incompleta entre puntos válidos.

7.1 Referencias de una zona

createZone() normaliza nombre, color, tipo, visibilidad, cierre y referencia. Las referencias admitidas son station, point y custom.

Tabla 8. Tipos de referencia admitidos por una zona.

Referencia	Comportamiento
station	Usa E0/N0; corresponde al BM del levantamiento.
point	Usa una observación calculada de otra zona como origen.
custom	Usa coordenadas Este/Norte escritas en el formulario de zona.

Estado geométrico

Las métricas solo se publican cuando la zona cumple sus condiciones de completitud y la referencia seleccionada produce una base válida.

8. Puntos: validación y ciclo de vida

La validación ocurre en el formulario modal, en la edición de tabla y durante `computeSurvey()`. Los controles visuales impiden valores fuera de rango y el motor vuelve a comprobarlos antes de asignar coordenadas.

Tabla 9. Reglas de validación de una observación.

Campo	Regla
Número	Entero positivo y único dentro de la zona.
Grados	$0 \leq G \leq 360$; con $G = 360$, $M = 0$ y $S = 0$.
Minutos	$0 \leq M < 60$.
Segundos	$0 \leq S < 60$.
Distancia	$d \geq 0$; para producir coordenadas debe ser mayor que 0.
Descripción	Texto libre; se protege en los archivos CSV cuando contiene comas o comillas.

Operaciones sobre una observación

- `normalizeObservation()` convierte tipos y limita ángulos.
- `moveObservation()` cambia el orden dentro de la zona y renumera.
- `clearPointValues()` conserva UID, zona y número, pero reemplaza los demás valores.
- `deleteObservation()` elimina el elemento del array, renumera y mantiene cerrado el polígono.
- `buildInputErrors()` detecta repetidos, secuencia, duplicados de coordenada y geometría cruzada.

Invariante

Una fila solo participa en la geometría cuando tiene ángulo válido y distancia mayor que 0. Una fila vacía puede permanecer para captura posterior sin crear conexiones incorrectas.

9. Plano: renderizado cartesiano

Código 8. Plano técnico exportable - app.js

```
1954     <span>Área útil: <strong>${formatNumber(details.usefulWidth, 0)} mm × ${formatNumber(details.usefulHeight, 0)} mm</strong></span>
1955     <strong class="drawing-scale-result ${details.fits ? "fits" : "does-not-fit"}">${escapeHtml(fitMessage)}</strong>`;
1956   }
1957
1958   function createTechnicalPlanCanvas(survey, options = {}) {
1959     const canvas = document.createElement("canvas");
1960     canvas.width = 1400;
1961     canvas.height = 820;
1962     const ctx = canvas.getContext("2d");
1963     const width = canvas.width;
1964     const height = canvas.height;
1965     const station = { east: toNumber(state.station.east), north: toNumber(state.station.north), id: "BM" };
1966     const exportZones = state.zones.filter((zone) => zone.visible && (!options.focusZoneId || zone.id === options.focusZoneId));
1967     const visibleZoneIds = new Set(exportZones.map((zone) => zone.id));
1968     const visiblePoints = survey.points.filter((point) => point.hasCoordinates && visibleZoneIds.has(point.zoneId));
1969     const eastValues = [station, ...visiblePoints].map((point) => point.east);
1970     const northValues = [station, ...visiblePoints].map((point) => point.north);
1971     const axisBuilder = options.tightFraming ? fittedAxis : niceAxis;
1972     let eastAxis = axisBuilder(Math.min(...eastValues), Math.max(...eastValues), station.east);
1973     let northAxis = axisBuilder(Math.min(...northValues), Math.max(...northValues), station.north);
1974     const margin = { left: 100, right: 55, top: 78, bottom: 85 };
1975     const plotW = width - margin.left - margin.right;
```

Figura 6. Fragmento de código: Construcción del canvas técnico con salida blanca.

drawPlot() adapta la resolución del canvas al devicePixelRatio, calcula ejes balanceados y conserva la misma escala gráfica en Este y Norte. plotTransform almacena las funciones de conversión entre coordenadas y píxeles.

Tabla 10. Responsabilidades del motor de renderizado.

Componente	Responsabilidad
niceAxis / axisWithinRange	Generar límites y marcas de cuadrícula legibles.
balancedAxes	Igualar unidades por píxel y evitar deformación del plano.
applyView	Aplicar zoom y desplazamiento sin mover el BM lógico.
drawZoneGeometry	Dibujar polígonos y líneas por zona.
drawPoint / drawLegend	Puntos, BM, numeración y leyenda.
createTechnicalPlanCanvas	Crear PNG e imagen de informe a 1400 × 820 px, siempre blanca.

9.1 Herramientas rápidas del plano

- `pointNearCanvasEvent()` busca el punto visible más cercano en un radio de 20 px.
- `setQuickMode()` activa selección, movimiento, limpieza o medición.
- Mover abre el mismo formulario de edición utilizado por la tabla.
- Medir calcula la distancia euclidiana entre dos coordenadas existentes.
- El arrastre del canvas modifica `panEast` y `panNorth`; la rueda ajusta zoom entre 0.5 y 20.
- Los interruptores de puntos, líneas y cuadrícula alteran visualización, no el modelo geométrico.

Separación de responsabilidades

La posición de un punto no se modifica directamente por arrastre. Mover significa editar su observación, con lo cual se conserva la trazabilidad del cálculo.

10. Cálculos: conversión angular, rumbo y proyecciones

El motor normaliza los ángulos a una vuelta completa y expresa el azimut en notación cuadrantal para facilitar la revisión topográfica.

Código 5. Conversión GMS y rumbo - app.js

```
365     String(observation.description || "").trim() !== ""
366   );
367 }
368
369 function dmsToDecimal({ degrees, minutes, seconds }) {
370   return toNumber(degrees) + toNumber(minutes) / 60 + toNumber(seconds) / 3600;
371 }
372
373 function decimalToDms(value) {
374   const normalized = ((value % 360) + 360) % 360;
375   let degrees = Math.floor(normalized);
376   const minuteFloat = (normalized - degrees) * 60;
377   let minutes = Math.floor(minuteFloat);
378   let seconds = Math.round((minuteFloat - minutes) * 60000) / 1000;
379   if (seconds >= 60) {
380     seconds = 0;
381     minutes += 1;
382   }
383   if (minutes >= 60) {
384     minutes = 0;
385     degrees = (degrees + 1) % 360;
386   }
387   return { degrees, minutes, seconds };
388 }
389
390 function formatDms(value) {
391   const dms = decimalToDms(value);
392   return `${dms.degrees}\u00b0 ${String(dms.minutes).padStart(2, "0")}' ${dms.seconds.toFixed(3).padStart(6, "0")}";
393 }
394
395 function bearingFromAzimuth(azimuth) {
396   const angle = ((azimuth % 360) + 360) % 360;
397   if (angle <= 90) return `N ${formatDms(angle)} E`;
```

Figura 7. Fragmento de código: Conversión GMS, normalización y rumbo.

Tabla 11. Conversión del azimut de GMS a grados decimales.

Operación	Expresión
Azimut decimal	$Az = G + M/60 + S/3600$
Conversión a radianes	$Azrad = Az \times \pi / 180$
Rumbo bearingFromAzimuth() reduce el ángulo al cuadrante correspondiente y devuelve N/S, el ángulo reducido y E/O.	

10.1 Proyecciones y coordenadas cartesianas

Código 6. Proyecciones y coordenadas - app.js

```
495     });
496   }
497   const runtime = runtimes.get(zone.id);
498   const normalized = normalizeObservation(observation);
499   const angleValid =
500     toNumber(observation.degrees) >= 0 &&
501     toNumber(observation.degrees) <= 360 &&
502     toNumber(observation.minutes) >= 0 &&
503     toNumber(observation.minutes) < 60 &&
504     toNumber(observation.seconds) >= 0 &&
505     toNumber(observation.seconds) < 60 &&
506     !(toNumber(observation.degrees) === 360 && (toNumber(observation.minutes) !== 0 || toNumber(observation.seconds) !== 0));
507   const hasCoordinates = toNumber(observation.distance) > 0 && angleValid;
508   const azimuth = dmsToDecimal(normalized);
509   const radians = (azimuth * Math.PI) / 180;
510   const distance = Math.max(0, toNumber(normalized.distance));
511   const deltaEast = distance * Math.sin(radians);
512   const deltaNorth = distance * Math.cos(radians);
513   const base = state.mode === "poligonal" ? runtime.cursor : runtime.base;
514   const point = {
515     uid: observation.uid,
516     id: String(observation.id || ""),
517     zoneId: zone.id,
518     rowIndex,
519     azimuth,
520     radians,
521     distance,
522     deltaEast,
523     deltaNorth,
524     east: base.east + deltaEast,
525     north: base.north + deltaNorth,
```

Figura 8. Fragmento de código: Validación, proyecciones y coordenadas cartesianas.

Tabla 12. Fórmulas de proyección y coordenadas cartesianas.

Operación	Expresión
Proyección Este	$\Delta E = d \times \text{sen}(\text{Azrad})$
Proyección Norte	$\Delta N = d \times \text{cos}(\text{Azrad})$
Coordenada Este	$E = E_{\text{base}} + \Delta E$
Coordenada Norte	$N = N_{\text{base}} + \Delta N$

En radiación, la base corresponde a la referencia de la zona. En modo poligonal, runtime.cursor avanza a la coordenada calculada del punto anterior.

Orientación matemática

El azimut parte del Norte y crece en sentido horario; por ello la proyección Este usa seno y la proyección Norte usa coseno.

11. Importar y exportar: formatos y salidas

Exportación de coordenadas TXT PENZD - app.js

```
2394 function exportCoordinatesTxt() {
2395   const survey = computeSurvey();
2396   const points = survey.points.filter((point) => point.hasCoordinates);
2397   if (!points.length) {
2398     showToast("No hay puntos calculados para exportar.", true);
2399     return;
2400   }
2401   const sanitizeDescription = (value) => String(value ?? "")
2402     .replace(/[,\\r\\n\\t]+/g, " ")
2403     .replace(/\\s+/g, " ")
2404     .trim();
2405   const rows = points.map((point, index) => [
2406     index + 1,
2407     formatNumber(point.east),
2408     formatNumber(point.north),
2409     0,
2410     sanitizeDescription(point.description),
2411   ]);
2412   const content = rows.map((row) => row.join(",")).join("\\r\\n");
2413   downloadFile(content, `TOPORAY_${safeFileName(state.projectName)}_coordenadas.txt`, "text/plain;charset=utf-8");
2414 }
2415
```

Figura 9. Fragmento de código: TXT PENZD sin encabezado, con numeración global y descripción.

Tabla 13. Formatos de intercambio y salida.

Formato	Implementación
CSV completo	Incluye zona, color, tipo, observación, cálculo y estado. csvEscape() protege comillas, comas y saltos.
Importar CSV	Normaliza encabezados, admite columnas alternativas y reconstruye zonas y puntos.
TXT de coordenadas	Solicita confirmación previa, filtra puntos válidos, renumera desde 1 y produce PENZD sin encabezado ni espacios; Z es 0.
TXT cálculos	Describe conversión GMS, azimut, rumbo, proyecciones y coordenadas.
PNG	Convierte el canvas técnico a Blob de imagen/png.
Informe	Genera HTML de impresión en tres páginas: resumen, resultados completos y plano ampliado.

Codificación

El CSV completo conserva marca BOM UTF-8 para uso tabular. El TXT de coordenadas se genera en UTF-8 sin BOM, usa comas como separadores, punto decimal, orden PENZD, descripciones saneadas y omite encabezado y espacios de alineación para su importación en Civil 3D. El TXT de cálculos conserva el desarrollo técnico en texto legible.

12. Configuración: informe técnico y escala de dibujo

`drawingScaleDetails()` compara la extensión real del levantamiento con el área útil del papel. La escala automática selecciona el primer denominador normalizado que permite encajar el dibujo.

Tabla 14. Parámetros de escala e informe técnico.

Parámetro	Valor o regla
Formatos	A0 841×1189, A1 594×841, A2 420×594, A3 297×420 y A4 210×297 mm.
Margen	10 mm por lado.
Cajetín	35 mm reservados en la dimensión inferior.
Orientación	Vertical u horizontal; intercambia ancho y alto.
Escalas	1:50 a 1:10000 en denominadores predefinidos.
Plano exportable	Fondo blanco, ejes balanceados, leyenda y BM.
Informe	Carta horizontal sobre plantilla TOPORAY: resumen, tabla completa de 22 columnas y gráfica ampliada.

- La vista interactiva puede ser oscura o clara; la salida técnica siempre es blanca.
- El informe usa `escapeHtml()` para insertar nombres y descripciones.
- Los resultados de todas las zonas se integran en la segunda página sin exceder el ancho del papel.
- La tercera página usa encuadre ajustado para mostrar el terreno principal y todas las zonas visibles.
- `window.print()` permite seleccionar Guardar como PDF.
- El navegador debe permitir la ventana emergente del informe.

Verificación requerida

Después de cambiar la plantilla o CSS de impresión, genere un PDF real y renderícelo a imagen para comprobar recortes, fondos y legibilidad.

13. Créditos: estructura institucional

La vista #view-credits se declara directamente en index.html. No depende del estado del levantamiento y debe mantenerse disponible en todos los tamaños de pantalla.

Tabla 15. Componentes técnicos de la vista Créditos.

Elemento	Implementación
Contenedor	section.app-view con data-view-panel=credits.
Equipo	Tres article.credit-item dentro de .credits-grid.
Identificación	Nombre completo, título profesional y afiliación institucional.
Presentación	Tarjetas con los colores de identidad definidos en styles.css.
Navegación	Acceso desde data-view=credits en escritorio y desde el menú móvil Más.

- Mantenga la información institucional en HTML para que siga siendo accesible y seleccionable.
- Actualice nombres, títulos o afiliaciones solamente con información aprobada por el equipo.
- Compruebe contraste, ajuste de texto y distribución a 320 px antes de publicar cambios visuales.

Alcance

La vista Créditos es informativa. No modifica state, localStorage, cálculos ni archivos exportados.

14. Eventos, accesibilidad y comportamiento móvil

Código 12. Cambio de vista y menú adaptable - app.js

```
2659         <td>${formatNumber(point.north)}</td><td>${formatNumber(point.east)}</td>
2660         <td>${segment ? formatNumber(segment.distance) : ""}</td><td>${segment ? escapeHtml(segment.label) : ""}</td>
2661     </tr>';
2662     }).join("");
2663     const completeInitialRow = `<tr class="initial-row"><td>BM</td><td>Referencia</td><td>Coordenada inicial</td><td colspan="15"></td><td>${formatNumber(toNumber(state.station.n
2664     const popup = window.open("", "_blank", "width=1100,height=800");
2665     if (!popup) {
2666         showToast("El navegador bloqueó la ventana de impresión. Permita las ventanas emergentes e inténtelo de nuevo.", true);
2667         return;
2668     }
2669     popup.document.write(`<!doctype html><html lang="es"><head><meta charset="utf-8"><title>Informe_TOPORAY_${escapeHtml(safeFileName(state.projectName))}</title><style>
2670     @page{size:letter landscape;margin:0}
2671     *{box-sizing:border-box}html,body{margin:0;background:#fff;font-family:Segoe UI,Arial,sans-serif;color:#111}body{-webkit-print-color-adjust:exact;print-color-adjust:exact}.
2672     </style></head><body><button class="print-command" onclick="window.print()">Imprimir o guardar como PDF</button>
2673     <main class="report-page summary-page">
2674         
2675         <section class="report-content">
2676             <h1>${escapeHtml(state.projectName)}</h1>
2677             <div class="project-data"><span><strong>Fecha del levantamiento:</strong> ${escapeHtml(formatDateValue(state.projectMeta.surveyDate))}</span><span><strong>Responsable:</strong>
```

Figura 10. Fragmento de código: Cambio de vista y comportamiento del menú según el ancho.

Tabla 16. Prácticas de eventos, accesibilidad y adaptación móvil.

Área	Práctica implementada
Eventos	Listeners centralizados al final de app.js; cada función modifica state y llama update().
Teclado	Botones, selects, inputs y dialog nativos conservan interacción estándar.
Etiquetas	aria-label en navegación, canvas, botones de icono y controles sin texto visible.
Mensajes	Toast con role=status y regiones aria-live para resultados dinámicos.
Responsive	Menú lateral móvil, navegación inferior y reorganización de formularios.
Tablas	Scroll horizontal con columnas de identificación y acciones fijadas.

Cuando se añadan controles nuevos, deben conservar un nombre accesible, estados disabled coherentes y áreas táctiles suficientes. No debe dependerse exclusivamente del color para comunicar errores.

15. Pruebas y control de calidad

Antes de publicar una versión se recomienda ejecutar la siguiente matriz mínima.

Tabla 17. Matriz mínima de pruebas funcionales.

Grupo	Casos obligatorios
Validación	G=0/360; M y S en 0, 59.999 y 60; distancia negativa, 0 y positiva; número repetido.
Cálculo	Azimuths en los cuatro cuadrantes; proyecciones con signos esperados; base BM y referencia de punto.
Geometría	Polígono de 3 y 4 puntos; línea; punto aislado; cruce; duplicado; vacío intermedio.
Puntos	Editar, limpiar, reingresar, eliminar, reordenar y deshacer.
Persistencia	Guardar, recargar, abrir, duplicar, renombrar y eliminar proyecto.
Intercambio	CSV completo; importación; TXT PENZD secuencial sin BOM; Z=0; descripción por punto.
Visual	Canvas cuadrado, BM fijo, zoom, arrastre, escritorio y 320/430/980/1600 px.
Salida	PNG blanco 1400×820; informe PDF de tres páginas sin recortes ni fondos negros.

Comprobaciones automáticas recomendadas

- Ejecutar `node --check app.js`.
- Usar Playwright para navegar todas las vistas y registrar `pageerror`.
- Interceptar descargas y validar nombres, columnas y codificación.
- Leer píxeles de la gráfica exportada para comprobar fondo blanco.
- Renderizar el PDF de informe y revisar visualmente cada página.

16. Mantenimiento, seguridad y limitaciones

Mantenimiento

- Conserve STORAGE_KEY y las claves de proyectos o implemente una migración explícita.
- Actualice migrateState() cuando agregue propiedades persistentes.
- Mantenga computeSurvey() libre de efectos secundarios para facilitar pruebas.
- Añada formatos de exportación como funciones separadas que consuman computeSurvey().
- Pruebe la plantilla de informe cada vez que cambien columnas o tipografías.
- Actualice la versión visible, README y manuales en una misma entrega.

Seguridad y privacidad

- La aplicación no transmite datos por sí sola y no contiene autenticación.
- escapeHtml() debe usarse para cualquier texto insertado en el informe HTML.
- Los archivos importados deben tratarse como datos no confiables y validarse.
- Un despliegue institucional debe usar HTTPS y políticas de contenido apropiadas.

Limitaciones conocidas

Tabla 18. Limitaciones técnicas conocidas.

Limitación	Implicación
Modelo 2D	Z se exporta como 0; no hay cálculo altimétrico.
Plano local	No hay transformación de datum, proyección cartográfica ni geodesia.
Persistencia local	No hay colaboración, cuentas, sincronización ni respaldo remoto.
Precisión	Los cálculos usan Number de JavaScript; el redondeo visible no cambia el valor interno.
Informe por navegador	El resultado final depende parcialmente del motor de impresión.

Criterio de liberación

Una versión debe publicarse solo después de validar cálculo, persistencia, exportaciones, comportamiento responsive y representación PDF con datos de prueba conocidos.